ProEm Instruction Set

Draft v0.7.1

Paul Vincent Bezzina

14 October 2026

1 Summary

The RP2040 programs the emulator with a short list of byte instructions. The control unit decodes each instruction into one write of the frame chain's configuration registers (`cfg_*`); there is no instruction memory and nothing is replayed after reset. A program describes one frame: which block comes first, how many bits each block handles, which way each block talks, and what ends the frame. A `RUN` instruction then starts it.

Everything in this draft is implemented in the RTL and tested end to end at pin level: each test sends instruction bytes to the chip, drives or models the protocol's real lines, and reads the results back over the return UART. v0.4 adds the line codec behind `LINE` and `CLOCK`, the `OUT` and `LINK` instructions, auto-baud and the return link (Section 14). The changes from 25 September:

1. **Block index and direction in every block instruction.** Each instruction names the block it configures (`HDR 1`, `HDR 2`, ...) and carries its direction bit (0 = read, 1 = write). This is how the RTL already works internally; the ISA now makes it explicit.
2. **Run modes and block groups** NEW. Each block is tagged *always*, *read only* or *write only*. `RUN READ` runs the *always* and *read only* blocks; `RUN WRITE` runs the *always* and *write only* blocks. One program then covers both an I^2C register read and a register write.
3. **END pair compare** NEW. END can combine two pins instead of watching one. With D+ and D− this detects the USB end of packet (both low) and flags the illegal state (both high).
4. **Line decoding** NEW. `LINE` selects how a pin is decoded before the chain sees it: plain, NRZI (USB) or Manchester (10BASE-T), with optional bit unstuffing.
5. **Clock source** NEW. `CLOCK` selects where the chain's `tick` comes from: SYNC (an input clock pin), ASYNC (an internal bit timer, as in a UART), generated by the chip on an output pin, or recovered by the line decoder.

Items marked NEW were added in v0.4 (25 September).

v0.5 (27 September). Items marked v0.5 are new in this version:

1. `BRANCH` (`0xF`) v0.5: two rules per chain that change the frame's direction, or abort it, depending on a field the chain has just received, or on the line not reading back what the chip drove. This adds I^2C target mode, SWD-style acknowledge handling and CAN-style arbitration.
2. **Capture any value** v0.5: a read header with `ANY` accepts whatever arrives and can return it, for fields such as the I^2C R/W bit, an SWD acknowledge or a USB PID.
3. **I^2C target** v0.5: with `OUT I2C` and a SYNC clock the codec detects START, repeated START and STOP, and drives SDA only while SCL is low.

4. **Streaming and underrun** v0.5: `DATA` can keep arriving while a write frame runs. On a line with fixed timing, running out of data aborts the frame and reports an underrun.
5. **Status byte** v0.5: bits 5–7 now report underrun, a branch match and a branch abort.

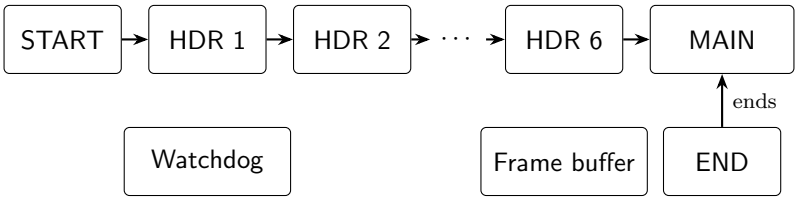
v0.6 (11 October). Items marked v0.6 are new in this version:

1. **CRC on chip** v0.6: `BRANCH 2` and `BRANCH 3` set up one CRC per chain, any width from 1 to 16 bits and any polynomial. On a read frame the chain checks the CRC as the bits arrive and returns the result; on a write frame it appends the CRC after MAIN (Section 5).
2. **Reply** v0.6: `BRANCH 4` sends a short fixed pattern (up to 16 bits) a set number of bit times after a read frame ends, optionally only when the CRC was good or bad. This is the USB handshake (ACK) and the CAN acknowledge slot.
3. **Branch on the CRC** v0.6: a rule with `SRC` = 1 and `BLOCK` = 1 acts on the CRC result.
4. **CAN bit stuffing tested** v0.6: `LINE STUFF` = 5 with `SANY` = 1 now has end-to-end tests, receive and chip to chip.

v0.7 (11 October). Items marked v0.7 are new in this version. They remove the limits found while testing fifteen further standards; every field was reserved before, so v0.6 programs work unchanged.

1. **22-bit divider** v0.7: `CLOCK` byte 2 bits 5–0 are `DIV[21:16]`. The slowest rate at 50 MHz falls from 763 to 12 bit/s (RC5 infrared, K-Line 5-baud init).
2. **Manchester boundary lock** v0.7: `LINE` byte 1 bit 3, `BND`, makes the decoder treat the first edge after idle as a bit boundary (DALI, whose start bit begins with a boundary edge).
3. **Seeking headers** v0.7: `HDR` byte 2 bit 5, `SEEK`, makes a read header skip incoming bits until its first bit appears, so a frame can wait out a gap of any length, for example idle time before a start bit (LIN inter-byte space).
4. **Byte-field CRCs** v0.7: `BRANCH 3` byte 1 bits 3 and 2, `BYTE` and `SPAN`, let the CRC skip fields shorter than 8 bits and run across frames (Modbus RTU, SMBus PEC).
5. **Release while reading** v0.7: with `DRIVE` = `01` the data and pair outputs are released during generated read bits and between entries of a generated-clock frame, so one push-pull line can carry both directions (3-wire SPI, MDIO).

2 Frame chain recap

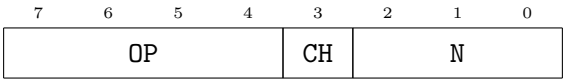


- Blocks run strictly in order, one bit per `tick`. A block that is not configured (or, with groups, not active in this run) is skipped.
- A *read* block samples its pin on each tick and compares or captures. A mismatch aborts the frame.
- A *write* block pushes its value into the frame buffer as a `wire` entry for the TX sender and succeeds in one clock.

- Read blocks only take ticks once every written bit ahead of them has left the chip (`line_free`), so a frame can write a request and read the reply.
- START and END have their own pins. The headers and MAIN share the data pin set by `MAIN`.
- Up to two chains (A and B) run side by side, for full-duplex SPI.

3 Instruction format

Every instruction starts with one byte: a 4-bit opcode and a 4-bit index. Operand bytes follow; multi-byte numbers are sent high byte first.



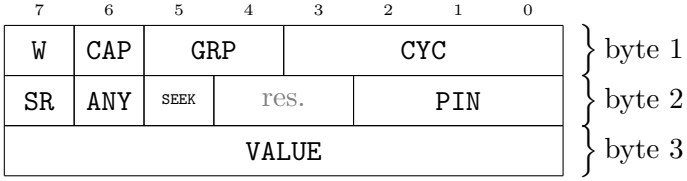
`CH` selects chain A (0) or chain B (1). `N` is the block number for `HDR` (1–6) or a small mode field for the other opcodes.

OP	Mnemonic	Operand bytes	Configures	RTL `cfg_sel`
0x0	NOP	0	–	–
0x1	START	3	START block	0
0x2	HDR n	3	Header block n = 1–6	1–6
0x3	MAIN	3	MAIN block and data pin	14
0x4	END	4	END block	15
0x5	WDOG	2	Watchdog limit	13
0x6	ORDER	0	Bit order, N[0]: 0 LSB, 1 MSB first	12
0x7	RUN	0	Start a frame, N[1:0] = mode NEW	–
0x8	HALT	0	Stop the chain and flush its buffer NEW	–
0x9	CLEAR	0	Unconfigure every block of the chain NEW	–
0xA	LINE	2	Line decoder for one pin NEW	–
0xB	CLOCK	4	Tick source NEW	–
0xC	DATA	1	Host entry, N = length − 1 NEW	–
0xD	OUT	1	Output drive, I^{2}C conditions, end of packet NEW	–
0xE	LINK	2	Return link divider (global) NEW	–
0xF	BRANCH	3	N = 0, 1: branch rule v0.5; 2, 3: CRC; 4: reply v0.6	8, 9, 10, 11, 7

4 Block instructions

4.1 START and HDR n

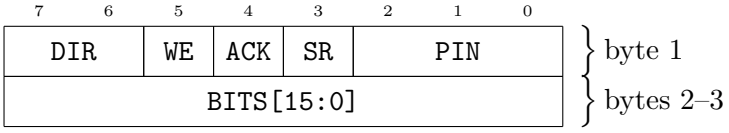
Both take the same three operand bytes.



Field	Meaning
`W`	Direction. 0 = read (compare against the line), 1 = write (send `VALUE`).
`CAP`	Read blocks only: also push the received value to the host.
`GRP`	NEW `HDR` only (START always runs). Group: `00` always, `01` read runs only, `10` write runs only, `11` reserved.
`CYC`	Field length in bits, 1–8. For `START` only, 0 means a *condition* START: succeed as soon as `PIN` equals `VALUE[0]`, without using a tick (SPI chip select). v0.5 In I²C target mode (`OUT I2C` with a SYNC clock) it waits for an I²C START or repeated START instead.
`SR`	NEW `HDR` only: a repeated START (Sr) just before this block. The chain sends the TX sender a `wire` entry of length 0, which means "Sr"; the block waits until it has gone out. Needed for I²C write-then-read.
`ANY`	v0.5 `HDR` read blocks only: accept any value instead of comparing. The received bits are kept, returned with `CAP`, and can be tested by `BRANCH`.
`SEEK`	v0.7 `HDR` read blocks only (not with `ANY`): ignore incoming bits until one equals the field's first bit, then compare the remaining `CYC`−1 bits as usual (a mismatch there still aborts). Use it to wait out a gap of unknown length in the middle of a frame: a seeking 1-bit header with `VALUE` 0 waits for the next start bit. The watchdog still ends a frame that never continues.
`PIN`	`START` only: its own input pin. Headers ignore it and use the data pin set by `MAIN`.
`VALUE`	The value to send or compare, as a plain number in the bottom `CYC` bits. The control unit handles bit order and alignment.

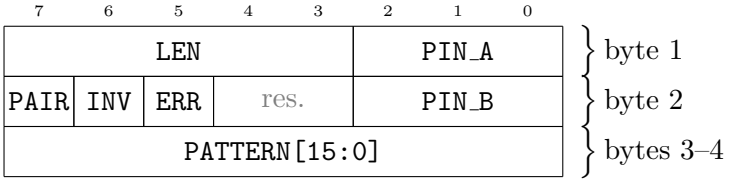
Headers must be configured in order from 1; the first unconfigured header ends the header list.

4.2 MAIN



Field	Meaning
`DIR`	`00` read, `01` write, `10` NEW follow the run mode (read in `RUN READ`, write in `RUN WRITE`).
`WE`	Wait for END: finish when END matches instead of after `BITS` bits (read only).
`ACK`	Add one bit in the other direction after every byte. Reading: send 0, or 1 after the last byte. Writing: listen, and end the frame with host entry `{0,02}` (NACK) on a 1.
`SR`	NEW Repeated START before MAIN, as for `HDR`.
`PIN`	Data pin, used by MAIN and every header.
`BITS`	Data bits per frame, 1–65535.

4.3 END



END shifts one input bit into a 16-bit window on every `end_tick` and matches the last `LEN` bits (1–16) against `PATTERN`. It ends a MAIN running with `WE`, and after an abort it makes START wait until END has matched again. `PAIR`, `INV`, `ERR` and `PIN_B` are NEW.

Field	Meaning
`PAIR` = 0	Single pin: the window input is `PIN_A`. Used for 10BASE-T (raw line, pattern of all ones).
`PAIR` = 1	Pair compare: the window input is built from `PIN_A` and `PIN_B` (Section 7).
`INV`	Invert `PIN_B` before the compare.
`ERR`	Abort the frame and raise `line_error` when both inputs are 1 on an `end_tick`.

4.4 DATA

`DATA` puts one {`length, value`} entry into the chain's 8-entry host FIFO, which feeds a write MAIN. When the FIFO is full (`host_full`) further entries are dropped and reported as a link event, so the RP2040 watches `HOST_FULL` (or counts) and paces its entries. The length (1–8) is `N` + 1 and the value is the one operand byte, so each entry costs two bytes on the serial link. Entries can be loaded before `RUN`.

Streaming v0.5. `DATA` can keep arriving while a write frame runs, so a frame is no longer limited to what fits in the FIFO. What happens when MAIN runs out of data depends on the clock:

- **GENERATE** (SPI controller, I²C controller, SWD): the generated clock simply pauses until the next entry arrives, which every clocked protocol allows. The watchdog still ends a frame that stalls for longer than `WDOG`.
- **ASYNC, RECOVER, SYNC** (UART, USB, Manchester, SPI target, I²C target): the line has fixed timing or the other side owns the clock, so a gap would corrupt the frame. If MAIN needs data, none is waiting and every bit already queued has gone out, the frame aborts, the chain stops driving, and the status byte has the abort and *underrun* bits set (`0x22`).

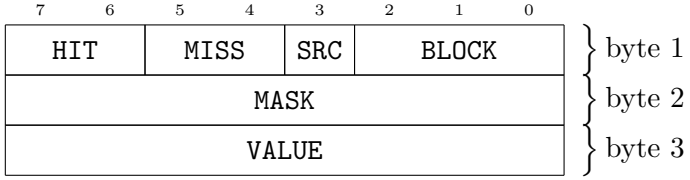
Each `DATA` entry takes 20 bit times on the instruction link, 140 clocks at 7 clocks per bit. A fixed-timing line keeps up if each entry lasts longer than that on the line: an 8-bit entry needs at least 18 clocks per bit. For a UART sent as three entries per byte (data, stop bit, next start bit) that is at least 42 clocks per bit, about 1.2 Mbaud at 50 MHz. Load a few entries before `RUN WRITE` so the frame starts with some margin.

4.5 WDOG, ORDER

`WDOG` takes a 16-bit limit in clocks; the frame ends if no bit or host entry arrives for that long (0 = off). `ORDER` sets LSB or MSB first for every block of the chain and should come before `END`.

4.6 BRANCH v0.5

`BRANCH` lets a frame decide its own path from what it receives. Each chain has two rules, chosen by `N` (0 or 1). v0.6 `N` = 2, 3 and 4 configure the CRC and the reply (Section 5); `N` > 4 raises `cmd_error`.



Field	Meaning
HIT, MISS	Action on a match and on a miss: 00 keep the current mode, 01 switch to read, 10 switch to write, 11 abort the frame.
SRC	0: test a block's value when it succeeds. 1: an event on the line, chosen by BLOCK.
BLOCK	SRC = 0: the block to watch, 0 = START, 1–6 = HDR n. SRC = 1: 0 = fire on the first bit where the line reads back differently from what the chip drove (arbitration; only HIT is used); v0.6 1 = fire when the CRC check finishes, HIT if the CRC was good, MISS if it was bad.
MASK, VALUE	SRC = 0: the rule matches when the watched block's value equals VALUE in every bit set in MASK. Values use the same convention as HDR (a plain number in the bottom bits).

- **Mode switch.** "Read" and "write" set the run mode for the rest of the frame, exactly as RUN READ and RUN WRITE would have: blocks with GRP = 01 or 10 run or are skipped accordingly, and a MAIN with DIR = follow takes that direction. The next frame starts again in the mode RUN gave.
- **Timing.** A rule is evaluated in the clock after its block succeeds; the chain pauses for that one clock, so the next block always starts in the new mode. Every standard gives at least 2 clocks per bit, so no bit is missed.
- **Abort.** A branch abort keeps the values captured so far (so the host sees, say, the acknowledge that caused it), stops the chain driving the line at once, discards any queued bits of the frame, and reports status 0x82 (0xC2 if it was the HIT action).
- **Read-back (SRC = 1, BLOCK = 0).** LINE's PIN_A must be the pad the chip drives. The comparison is made in the middle of each bit (ASYNC, RECOVER), in the middle of the high phase (GENERATE) or on the sampling edge (SYNC). The input pipeline delays the read-back by 3 clocks, so ASYNC needs at least 8 clocks per bit. While such a rule is set, a write frame stays open until its last bit has left the pin, so a late mismatch still reaches its status.
- **Two rules.** Rule 0 has priority when both act in the same clock. BRANCH with byte 1 = 00 turns a rule off; CLEAR turns both off.
- **Status.** Bit 6 of the frame's status byte records that a rule matched (took its HIT action); bit 7 that a branch aborted the frame.

5 CRC and reply v0.6

Each chain has one CRC unit and one reply pattern, set with three more BRANCH indices. All three take the usual three operand bytes.

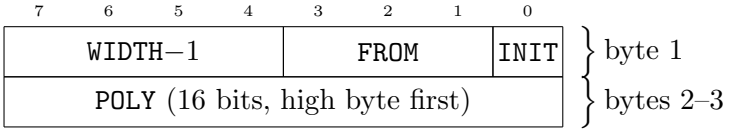
5.1 How the CRC is computed

The CRC is computed over the bits in the order they cross the wire, after bit unstuffing on receive and before bit stuffing on transmit. A register of WIDTH bits starts at all zeros or all ones; for each bit b, the feedback is f = (top bit of the register) XOR b, the register shifts left by one, and if $f = 1$ the polynomial is XORed in. The polynomial is written without its top term: CRC-16 ($x^{16} + x^{15} + x^2 + 1$) is 0x8005, CRC-5/USB ($x^5 + x^2 + 1$) is 0x05, CRC-15/CAN is 0x4599.

Working in wire order means ORDER does not change the CRC set-up: a standard that sends LSB first (USB) and one that sends MSB first (CAN) use the same instruction, and the catalogue value for a "reflected" CRC is simply what the register holds when the bits are taken in wire order.

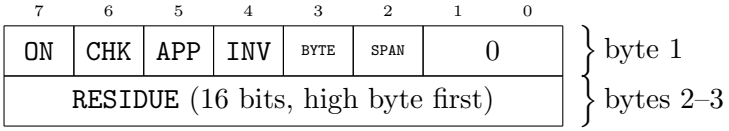
- **Receive.** Every bit the chain reads from `FROM` onward, including the received CRC bits themselves, goes through the register. At the end of MAIN the register holds the *residue*: 0 for a plain CRC, a fixed constant when the sender inverts its CRC (USB: `0x800D` for CRC-16, `0x0C` for CRC-5). The check passes when the register equals `RESIDUE`.
- **Transmit.** Every bit the chain writes from `FROM` onward goes through the register. After MAIN the chain sends the register, top bit first, inverted if `INV` = 1, as part of the same packet (stuffed like the data, before the end of packet).

5.2 `BRANCH 2`: CRC A



Field	Meaning
`WIDTH−1`	CRC width minus one: 0–15 for 1 to 16 bits.
`FROM`	First block covered: 0 = START, 1–6 = `HDR n`, 7 = MAIN only. Every later block is covered too. USB and Ethernet cover MAIN only (the PID and preamble are outside the CRC); CAN covers everything from the start-of-frame bit (`FROM` = 0).
`INIT`	0: the register starts at all zeros. 1: all ones.
`POLY`	The polynomial without its top term, right aligned.

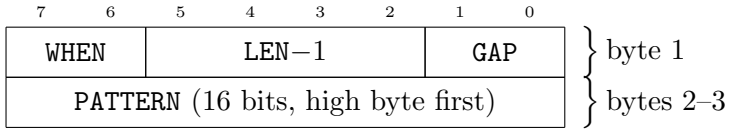
5.3 `BRANCH 3`: CRC B



Field	Meaning
`ON`	Turns the CRC unit on. `BRANCH 3` with byte 1 = `00` turns it off.
`CHK`	Check on read frames: return the result (below) after MAIN.
`APP`	Append on write frames: send the CRC after MAIN.
`INV`	Send the CRC inverted (USB). Has no effect on the check.
`BYTE`	v0.7 Only 8-bit fields go through the CRC: write entries of 8 bits, read blocks with `CYC` = 8, and `MAIN` when it is at least 8 bits long. Start, stop, parity and acknowledge bits are skipped, which is what byte-oriented CRCs such as Modbus and the SMBus PEC need.
`SPAN`	v0.7 Do not restart the CRC at the start of each frame; it runs on until `BRANCH 2` or `BRANCH 3` is written again. With one frame per UART character this covers a whole message; the host rewrites `BRANCH 3` before each message, and the result after the last character is the message's check.
`RESIDUE`	The register value that means "good" on receive, right aligned.

Result. With `CHK` = 1, a read frame returns one more host entry after its data and before its status: first byte `8E` (chain A) or `AE` (chain B), value bit 0 = CRC good, bit 1 = CRC checked. A good CRC is `03`, a bad one `02`. If the frame aborts before MAIN ends there is no result entry, only the status.

5.4 BRANCH 4: reply



Field	Meaning
WHEN	00 off, 01 reply when the CRC was good, 10 when it was bad, 11 always.
LEN−1	Pattern length minus one: 1 to 16 bits.
GAP	Bit times to wait after the frame's last bit before the reply starts: 0–3.
PATTERN	The bits to send, a plain number in the bottom LEN bits, sent in the chain's ORDER like DATA.

- The reply is a separate packet: on USB it is NRZI coded, stuffed and closed with its own end of packet, exactly as DATA would be; on CAN it is driven straight into the acknowledge slot. Put any SYNC or PID the reply needs into PATTERN.
- **Timing.** The gap is counted on the receive bit grid: the reply starts on a bit boundary of the frame just received, GAP bits after its last bit (one bit later if a stuff bit is due there, as in CAN). This needs a timed clock source (ASYNC or RECOVER) and a non-Manchester line; on other lines the gap is skipped.
- The reply and the appended CRC go straight to the line codec, so they are never held up by received data still waiting for the return link.
- CLEAR turns the CRC unit and the reply off.

5.5 Examples

Use	Bytes (chain A)	Notes
USB DATA receive, CRC-16	F2 FF 80 05 F3 C0 80 0D	16 bits, MAIN only, init ones; residue 0x800D. Capture the PID with a header (ANY) so that MAIN holds only the data and the CRC.
USB ACK after a good packet	F4 7F D2 80	WHEN good, 16 bits, gap 3: SYNC 0x80 and PID 0xD2, LSB first.
USB token, CRC-5	F2 4F 00 05 F3 C0 00 0C	5 bits, init ones, residue 0x0C.
USB DATA transmit	F2 FF 80 05 F3 B0 00 00	Append, inverted.
CAN receive, CRC-15	F2 E0 45 99 F3 C0 00 00	From START (the start-of-frame bit), init zeros, residue 0. With ORDER MSB.
CAN acknowledge	F4 44 00 02	WHEN good, 2 bits, gap 0: the CRC delimiter (1) then a dominant ACK (0).
CAN transmit	F2 E0 45 99 F3 A0 00 00	Append, not inverted.

6 Line decoding and clock source

A chain needs two things besides its blocks: a bit on its data pin and a tick saying when that bit is valid. LINE sets how the bit is produced, CLOCK sets where the tick comes from. Both apply to the chain named by CH; N is unused.

6.1 LINE

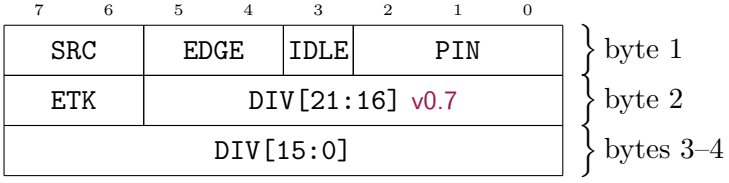


Every input pin passes through a two-flop synchroniser and one more register before any block sees it. With a timed clock source (ASYNC or RECOVER) the decoder works on pin `PIN_A` and its output *replaces* that pin for START, the headers and MAIN: a block whose `PIN` equals `PIN_A` reads decoded bits, one per `tick`. END and the other pins still see the raw lines, so END can watch the real end-of-frame state (the idle high of 10BASE-T, SE0 on USB). With SYNC or GENERATE the pins are used as they are.

`CODE`	Decoding	Used by
`000`	None: the pin is the bit	UART, SPI, I^2C, PS/2
`001`	NRZI: no change = 1, change = 0	Single-ended NRZI
`010`	NRZI on a differential pair: J/K from `PIN_A`, `PIN_B`; no bits while SE0 or SE1	USB (`PIN_A` = D+, `PIN_B` = D−)
`011`	Manchester, IEEE 802.3: low-to-high mid-bit = 1, high-to-low = 0	10BASE-T
`100`	Manchester, G. E. Thomas: the opposite convention	
`101`–`111`	Reserved	

- `INV` inverts the pin before decoding and the output after encoding (for USB full speed, where J is D+ high).
- v0.7 `BND` (Manchester only): after the line has been idle for two bit times, the decoder takes the first edge as a bit *boundary* and locks onto the middle edge half a bit later. Use it when frames start with a bit whose first half differs from the idle level, such as DALI's start bit. Without it the first edge is taken as a middle edge, which is right for frames that start with a preamble (10BASE-T) or with a bit whose first half equals idle (RC5).
- `STUFF` = n removes the bit after n equal bits in a row, and gives it no `tick`, so the chain never sees it; 0 turns unstuffing off. USB: $n = 6$, counting 1s only (`SANY` = 0). CAN would use $n = 5$ counting runs of either value (`SANY` = 1). A stuffed bit with the wrong value aborts the frame with `line_error`.
- The encoder uses the same setting for what the chain sends: NRZ, NRZI (starting from J), or Manchester, with a stuff bit inserted after n equal bits. With code `010` it drives the pair: D+ on the data output, D− on the pair output.
- NRZ and NRZI sample in the middle of each bit and re-centre on every edge. Manchester decodes on the mid-bit edges: after each accepted edge it ignores edges for $\frac{3}{4}$ of a bit, so boundary edges between equal bits are skipped. This needs at least 5 clocks per bit: at 4, an edge can sit exactly between the two cases.

6.2 CLOCK



SRC	Mode	Where tick comes from	Fields used
00	SYNC	Edges of an input clock pin (SCK, SCL, PS/2 clock)	PIN = clock input, EDGE
01	ASYNC	The decoder's bit timer on LINE's PIN_A, sampling half a bit after each edge; transmit runs the same timer freely	DIV = clocks per bit
10	GENERATE	The chip makes the clock on the clock output; one tick per bit, on the edge EDGE names	PIN = clock read-back input (clock stretching with open drain), EDGE, IDLE, DIV = clocks per half period (≥ 4)
11	RECOVER	As ASYNC; the name used with coded lines (NRZI, Manchester)	DIV = nominal clocks per bit

- EDGE: 00 rising, 01 falling, 10 both. This covers the SPI modes: sample on one edge; the encoder changes the output on the other (SYNC) or at the start of each bit (GENERATE).
- IDLE: level of a generated clock between frames (SPI CPOL; 1 for I^{2}C SCL).
- A generated clock only runs while there is work: the TX sender has a bit waiting or a read block is waiting for one (listen). It never runs past the data.
- ETK selects end_tick: 00 same as tick, 01 every system clock, 10 the decoder's mid-bit strobe (every bit time, including stuffed bits).
- v0.7 DIV is 22 bits: bytes 3–4 hold DIV[15:0] and byte 2 bits 5–0 hold DIV[21:16], so it reaches 4 194 303 clocks (12 bit/s at 50 MHz). Earlier programs leave those bits 0.
- DIV is in system clocks (50 MHz). UART at 115 200 baud: 434. USB low speed: 33 (1.515 Mb/s, within ±1.5 %). 10BASE-T receive: 5. I^{2}C at 100 kHz generated: 250 (half period). Manchester transmit needs an even DIV, so 10BASE-T transmit needs a 40 or 60 MHz clock.
- In I^{2}C mode (OUT) a generated SCL is low for the first half of each bit and high for the second, whatever IDLE says; the bus idles high between transactions.

6.3 OUT

7	6	5	4	3	2	1	0	
DRIVE		IDLE	PAIR	EOP	I2C	CSH	r	byte 1

Each chain has a data output, a pair output, a clock output (each with its own output enable) and a select output.

Field	Meaning
`DRIVE`	`00` off (after reset and `CLEAR`), `01` push-pull while a packet is being sent (v0.7 released during generated read bits and between entries of a generated-clock frame, so a shared data line can be read), `10` open drain on data and clock (drive low for 0, release for 1), `11` push-pull always.
`IDLE`	Data level between frames with `DRIVE = 11` (1 for a UART line).
`PAIR`	Drive the complement on the pair output (USB D−).
`EOP`	At the end of each frame send an end of packet: two bit times of SE0 then one of J on a pair (USB), or two bit times high (10BASE-T).
`I2C`	With a generated clock (controller): send START before the first byte, Sr for a repeated-START entry, and STOP after each frame. v0.5 With a SYNC clock on SCL (target): detect START, repeated START and STOP on `LINE`'s `PIN_A` (SDA) and the clock pin (SCL); start driving a bit only after SCL falls, and release SDA after the falling edge that ends it. A STOP or repeated START ends a MAIN with `WE` normally; anywhere else in a frame it aborts the frame.
`CSH`	Select output active high (0: active low, as SPI chip select). It is active from the frame's START until its last bit has gone out.

The end of a frame is marked by the frame's status entry (below), which passes through the frame buffer in order behind the frame's last bits. STOP and the end of packet therefore always come after the frame's own bits and before the next frame's.

6.4 `LINK` and the return link

`LINK` (`0xE`, global, 2 operand bytes) sets the return UART's clocks per bit; 0 uses the rate measured by auto-baud. At 50 MHz, 4 clocks per bit is 12.5 Mbaud, which the RP2040's PIO can receive.

Auto-baud. After reset the first byte on the instruction line must be `0x55`. The chip times its first five falling edges (8 bit times), sets its receive rate from that, and does not pass the byte on.

Return format. Every message is two bytes, sent with 1 start and 1 stop bit:

First byte	Second	Meaning
`10 C 0 LLLL`	value	Host entry from chain `C`, length `L` (0–8; length 0 value `02` = NACK)
`10 C 0 1110`	result	v0.6 CRC result of a read frame on chain `C`: bit 0 good, bit 1 checked; sent after the data and before the status
`10 C 0 1111`	flags	End of a frame on chain `C`: bit 0 complete, 1 abort, 2 timeout, 3 line error, 4 overflow, v0.5 5 underrun, 6 branch matched, 7 branch abort
`11 000000`	flags	Link events: bit 0 `rx_error`, 1 `cmd_error`, 2 `DATA` dropped on A, 3 on B

A frame whose host entries arrive faster than the return link can carry them fills the 16-entry frame buffer; further entries are dropped and the frame's status has the overflow bit set. Written bits are never dropped: write blocks wait for space.

7 END pair compare

With a = `pins[PIN_A]` and b' = `pins[PIN_B]` $\oplus$ `INV`:

$$\begin{aligned} \text{both_high} &= a \wedge b' \\ \text{equal} &= \neg(a \oplus b') \\ \text{window input} &= \text{equal} \wedge \neg\,\text{both_high} \;=\; \neg a \wedge \neg b' \\ \text{line_error} &= \texttt{ERR} \wedge \text{both_high} \quad \text{(checked on } \texttt{end_tick}\text{)} \end{aligned}$$

For USB low speed, a = D+, b = D−, `INV` = 0:

D+	D−	AND	XNOR	Window input	Line state
0	1	0	0	0	J (idle, data)
1	0	0	0	0	K (data)
0	0	0	1	**1**	SE0: end of packet
1	1	1	1	0	SE1: illegal, `line_error`

- `end_tick` must be the decoder's mid-bit sample strobe, not every clock. D+ and D− do not switch at exactly the same instant, so for a few nanoseconds around each J/K change both can read low (or high). Sampling mid-bit ignores that.
- `LEN` = 2, `PATTERN` = `0x0003`: END matches after two bit times of SE0, which is exactly what the sender drives.
- `end_tick` runs on every bit time, including stuffed bits, because it comes from the bit timer rather than from the unstuffed data `tick`.
- With `INV` = 1 the window input is 1 when `PIN_A` is low and `PIN_B` is high (for USB low speed with D+, D−: the J idle state), so END can wait for the line to return to idle; `ERR` should then be off.
- The pair compare looks at levels, so it cannot see the I^2C STOP condition, which is an edge (SDA rising while SCL is high). STOP and START detection stay in the pin block.

This replaces the separate `se0` pin proposed earlier: END reads the raw D+ and D− directly, while `LINE` gives the blocks the decoded bits.

8 Run modes and block groups

`RUN` starts a frame on the chain named by `CH`. `N[1:0]` selects the mode:

`N[1:0]`	Mode	Blocks that take part
`00`	`RUN`	`GRP = 00` only
`01`	`RUN READ`	`GRP = 00` and `01`; MAIN with `DIR` = follow reads
`10`	`RUN WRITE`	`GRP = 00` and `10`; MAIN with `DIR` = follow writes

- A chain whose START writes starts one frame straight away (this replaces the `go` input). In plain `RUN` mode it then also starts a new frame whenever host data is waiting, which is what a UART TX stream needs. In `RUN READ` and `RUN WRITE` it only starts on `RUN`, so `DATA` for the next transaction can be loaded early.
- A chain whose START reads is armed and waits for its pattern or condition; it stays armed after each frame.

- The mode stays set until the next `RUN`, so a listening chain keeps re-arming in the same mode. `RUN` is meant for between frames.
- `HALT` stops the frame in progress, drops anything not yet released from the frame buffer, empties the host FIFO and disarms the chain. `CLEAR` resets the chain completely: every block, both buffers, the bit order and its `LINE` and `CLOCK` settings.
- The router already skips unconfigured blocks. Groups add one more condition to that skip: the block's group must be active in this run. The per-block `W` bit is unchanged.

Why groups rather than "activate every READ block". A single I^2C transaction mixes both directions: the chip writes the address, reads the ACK, writes the register, reads the ACK, and so on. Every block's direction is fixed by the protocol, and the blocks must still run in order. So the direction must stay per block (the `W` bit). What changes between a register read and a register write is *which* blocks run, and which way MAIN goes. Groups and the follow mode for MAIN express exactly that, so the RP2040 loads one program and chooses the transaction with `RUN READ` or `RUN WRITE`.

9 Example programs

9.1 I^2C register read and write, one program

Device 0x48, register 0x05, chip as controller, chain A. SDA and SCL are open-drain pads: the chip drives them through the data and clock outputs and reads them back on pins 0 and 1.

Instruction	Bytes	W	Group	Job
`ORDER MSB`	`61`			MSB first
`LINE`	`A0 00 00`			No decoding
`CLOCK`	`B0 89 00 00 FA`			Generate SCL, 100 kHz, read back on pin 1
`OUT`	`D0 A4`			Open drain, I^2C conditions
`WDOG 4000`	`50 0F A0`			
`START`	`10 88 00 90`	1	always	Address 0x48 + W
`HDR 1`	`21 01 00 00`	0	always	ACK
`HDR 2`	`22 88 00 05`	1	always	Register 0x05
`HDR 3`	`23 01 00 00`	0	always	ACK
`HDR 4`	`24 98 80 91`	1	read	Sr, address 0x48 + R
`HDR 5`	`25 11 00 00`	0	read	ACK
`MAIN`	`30 90 00 10`	follow		16 bits, `ACK` on

- `RUN WRITE` (72) with two data bytes in the host FIFO: `S 90 A 05 A d0 A d1 A P`.
- `RUN READ` (71): `S 90 A 05 A Sr 91 A` then two bytes read, ACK then NACK, `P`.

9.2 USB low-speed receive

Chain A, LSB first. Pins: 0 = D+, 1 = D−. `LINE` decodes the pair onto pin 0 for the blocks; END reads the raw pins.

Instruction	Bytes	Job
`ORDER LSB`	`60`	
`LINE`	`A0 40 31`	NRZI pair, D+ = 0, D− = 1, unstuff after six 1s
`CLOCK`	`B0 C0 80 00 21`	Recover, 33 clocks per bit, `end_tick` = mid-bit strobe
`START`	`10 08 00 80`	Read 8 bits: SYNC `00000001`
`MAIN`	`30 20 FF FF`	Read on pin 0 until END
`END`	`40 10 A1 00 03`	Pair D+/D−, `ERR` on, 2 bits of SE0
`WDOG 200`	`50 00 C8`	
`RUN READ`	`71`	Arm

MAIN reads the PID as its first byte. v0.5 A header with `ANY` can now capture the PID instead, and a `BRANCH` can act on it. v0.6 The chip can now check the CRC-5 or CRC-16 itself and answer with an ACK (Section 5); the host still checks the PID complement.

9.3 USB low-speed transmit

Chain B sends SYNC, a DATA1 PID and data from the host FIFO, with bit stuffing and an end of packet:

Instruction	Bytes	Job
`LINE`	`A8 40 31`	NRZI pair, stuff after six 1s
`CLOCK`	`B8 C0 00 00 21`	33 clocks per bit
`OUT`	`D8 58`	Drive while sending, pair, end of packet
`START`	`18 88 00 80`	Write SYNC
`HDR 1`	`29 88 00 4B`	Write PID
`MAIN`	`38 40 00 20`	Write 32 bits from `DATA`
`DATA` × 4, `RUN WRITE`	`CF xx ... 7A`	Load the data, then send

9.4 Other standards

Standard	Program
UART RX	`LINE` none on RX; `CLOCK` ASYNC, `DIV` = 434; `START` read 1 bit = 0; `MAIN` read 9 bits. Host checks the stop bit.
UART TX	`CLOCK` ASYNC, `DIV` = 434; `OUT` push-pull always, idle 1; `START` write 1 bit = 0; `MAIN` write 9 bits; `DATA` {8, byte}, {1, 1}.
SPI controller	Chain A: `CLOCK` GENERATE; `OUT` push-pull; `START` write with `CYC` = 0; `MAIN` write on MOSI; chip select on the select output. Chain B: `CLOCK` SYNC on SCK read back, condition `START` on CS read back, `MAIN` read on MISO.
SPI target	Chain B reads MOSI with `CLOCK` SYNC on the master's SCK; chain A answers on MISO with `CLOCK` SYNC too, changing on the other edge.
PS/2 device to host	`CLOCK` SYNC, falling edge of the device clock; `START` read 1 bit = 0; `MAIN` read 10 bits.
10BASE-T RX	`LINE` Manchester IEEE; `CLOCK` RECOVER, `DIV` = 5, `end_tick` every clock; `START` read 8 bits `0xD5`; `HDR 1–6` compare the destination MAC; `MAIN` read with `WE`; `END` on the raw line, 8 ones; `LINK` 4.
Manchester TX	`LINE` Manchester; `CLOCK` RECOVER, even `DIV`; `OUT` drive while sending, `EOP`; preamble from `START` and six write headers, then `DATA`.

9.5 I²C target at address 0x42 v0.5

Chain A answers as a device at 0x42. SDA on pin 0, SCL on pin 1; the chain drives SDA open drain and never drives SCL. `HDR 2` captures the R/W bit, `HDR 3` sends the address ACK, and the branch turns MAIN into a writer (the controller reads) or a reader (the controller writes).

Instruction	Bytes	Job
`ORDER MSB`	`61`	
`LINE`	`A0 00 00`	SDA on pin 0 (also START/STOP detection)
`CLOCK`	`B0 01 00 00 00`	SYNC on SCL, pin 1, rising edge
`OUT`	`D0 84`	Open drain, I²C
`START`	`10 00 00 00`	Wait for an I²C START
`HDR 1`	`21 07 00 42`	Address 0x42 (a mismatch aborts: no ACK)
`HDR 2`	`22 41 40 00`	R/W bit, `ANY`, captured
`HDR 3`	`23 81 00 00`	Write the ACK (0)
`BRANCH 0`	`F0 92 01 01`	Watch `HDR 2`: R/W = 1 → write, else read
`MAIN`	`30 B0 00 40`	Follow, `WE`, `ACK`; ends at STOP or Sr
`RUN`	`70`	Arm (stays armed)

A write from the controller returns {`1,0`} (the R/W bit) and the bytes, then status `01`. A read sends the `DATA` entries loaded beforehand, returns {`1,1`} and {`0,02`} (the controller's final NACK), then status `41`. Tested from 417 kHz to 1 MHz SCL; above that the ACK after a repeated START can come late, because the previous frame's status entry must first reach the return link.

9.6 SWD-style acknowledge v0.5

Chain A as host, LSB first, SWCLK generated on the clock output (read back on pin 1), SWDIO open drain on pin 0. The request is written, the turnaround bit and the 3-bit acknowledge are read with `ANY`, and the branch continues to the 33-bit data phase only for OK (`001`).

```
60   A0 00 00   B0 81 00 00 19   D0 80   10 88 00 A5
21 01 40 00   22 43 40 00   F0 72 07 01   30 00 00 21   70 (per transaction)
```

OK returns {`3,1`}, four data bytes and the parity bit, then status `41`; WAIT or FAULT returns the acknowledge ({`3,2`} or {`3,4`}) and status `82`. The test uses an open-drain SWDIO model that samples on the rising edge of SWCLK; a real target drives push-pull, so check the turnaround timing against the target before relying on it.

9.7 CAN-style arbitration v0.5

Chain A sends a start-of-frame bit, an 11-bit identifier and data at 50 clocks per bit (1 Mbit/s) on an open-drain line read back on pin 0. A rule with `SRC` = 1 aborts the frame on the first bit where the chip sends recessive (1) but reads dominant (0).

```
61   A0 00 00   B0 40 00 00 32   D0 80   10 81 00 00   30 40 00 13   F0 C8 00 00
C7 id[10:3]   C2 id[2:0]   C7 data   72
```

A lost arbitration stops the chip driving within the same bit and reports `C2`; the winning frame continues undisturbed on the line and can be received by the other chain. The arbitration test runs without bit stuffing. v0.6 `LINE STUFF` = 5 with `SANY` = 1 is now tested on its own: a CAN frame with stuffing is received, its CRC-15 checked, and the acknowledge driven into the ACK slot, both from a node model and chip to chip.

10 Corrections in v0.7.1

No instruction changes. Two behaviours now match what this document already describes, and one timing detail changed to close timing at the slow corner.

- `CLOCK` GENERATE with an `EDGE` that samples on the trailing edge (SPI modes 1 and 3): the data output now changes on the leading edge, half a clock period before the device samples, instead of on the trailing edge itself. Modes 0 and 2 and I^2C are unchanged.
- A frame abort that cancels queued bits (`BRANCH` abort, underrun, I^2C target abort) can no longer lose a bit-done report when a second cancel or the next frame follows within two clocks. Before, the channel could stop listening until `CLEAR`.
- Timing: the CRC unit now updates one system clock after the bits it is fed, so the CRC result entry, an appended CRC and a reply start one clock (20 ns) later after the last bit of a frame. Reply gaps and acknowledge slots are counted in bit times, so no standard in the guide is affected.

11 Changes in v0.7

- `CLOCK`: `DIV` widened to 22 bits using the reserved bits of byte 2.
- `LINE`: byte 1 bit 3 is `BND`, the Manchester boundary lock.
- `HDR`: byte 2 bit 5 is `SEEK`.
- `BRANCH 3`: byte 1 bits 3 and 2 are `BYTE` and `SPAN`.
- `OUT`: `DRIVE` = `01` releases the data and pair outputs during generated read bits.
- Sampling note: with a generated push-pull clock, input bits are sampled about one system clock before the chip's own sampling edge; with an open-drain clock they are sampled once the clock is seen high, as I^2C clock stretching requires. Protocols that need data held only up to the edge (MDIO) should therefore use a push-pull clock, which `DRIVE` = `01` now allows together with a readable data line.

12 Changes in v0.6

- `BRANCH N` = 2, 3, 4 configure the CRC unit and the reply (RTL `cfg_sel` 10, 11 and 7); `cmd_error` now comes from `N` > 4.
- New return entry `8E`/`AE`: the CRC result, before the frame's status.
- `BRANCH SRC` = 1 with `BLOCK` = 1 acts on the CRC result.
- The appended CRC, the reply gap and the reply go straight to the line codec instead of queuing behind received data in the frame buffer.
- A wire entry of length 0 with bit 7 set is a wait of bits 6–0 bit times, counted on the receive bit grid (used for the reply gap).
- Bit unstuffing only starts counting inside a frame, so idle bits before a CAN start-of-frame no longer cause it to be taken for a stuff bit, and a stuff error is not reported while the chain is driving the line itself.

13 Changes in v0.5

- New `BRANCH` (`0xF`) with two rules per chain; the opcode was reserved.
- `HDR` byte 2 bit 6 is `ANY` (capture any value).
- I^2C target mode: START/STOP detection and SDA timing in the codec; a condition START waits for an I^2C START.
- `DATA` streams during a write frame; fixed-timing lines abort with an underrun when data runs out.
- Status bits 5 (underrun), 6 (branch matched), 7 (branch abort).
- `cmd_error` now comes from `BRANCH` with `N` > 1 or `HDR 0`/`HDR 7`; no opcode is reserved any more.

14 Changes in v0.4

- Line codec per chain behind `LINE` and `CLOCK`: NRZ, NRZI, NRZI pair, Manchester, bit stuffing both ways, all four clock sources.
- New `OUT` (`0xD`): drive mode, idle level, pair output, end of packet, I^2C START/Sr/STOP, select polarity. Outputs are off after reset.
- New `LINK` (`0xE`) and a return UART carrying host entries, a status per frame and link events.
- Auto-baud on the instruction line; the first byte after reset is `0x55`.
- In GENERATE mode `PIN` is the clock read-back input used for clock stretching.
- Write blocks wait when the frame buffer is almost full; received entries that do not fit are dropped and reported.

15 Changes in v0.3

- New `DATA` instruction (`0xC`) carries write data over the same serial link; the pin block moves to `0xD`–`0xF`.
- `GRP` and `SR` apply to headers and MAIN only; START ignores them.
- Repeated START is a `wire` entry of length 0 to the TX sender. A write START with `CYC` $=$ 0 (used for SPI) sends nothing.
- `RUN READ` and `RUN WRITE` start a write-START chain only on `RUN`; plain `RUN` also starts frames when host data waits.
- Corrected the meaning of END with `INV` $= 1$, and dropped the capturing header from the USB example.
- `cmd_error` pulses for a reserved opcode (none since v0.5) or `HDR 0`/`HDR 7`; `rx_error` pulses when a byte's stop bit is low.

16 Status and open items

Feature	State	Note
Frame chain, control unit, groups, `RUN` modes, END pair, `SR`, `HALT`, `CLEAR`, `DATA`	In RTL, tested	1,800 random programs at pin level plus the tests below
Line codec: NRZ, NRZI pair, Manchester, stuffing, SYNC, ASYNC, GENERATE, RECOVER	In RTL, tested	UART, SPI, I^2C, PS/2, 10BASE-T RX, Manchester TX, USB low speed RX and TX
`OUT`, `LINK`, auto-baud, return link	In RTL, tested	Every test reads its results back over the return UART
`BRANCH`, `ANY`, I^2C target, streaming and underrun v0.5	In RTL, tested	I^2C target, SWD-style and CAN-style tests, 600 random branch programs, cocotb controller-to-target loopback
CRC check and append, reply, branch on CRC, CAN stuffing v0.6	In RTL, tested	USB CRC-16 and CRC-5 with ACK, USB transmit with CRC, CAN CRC-15 with ACK from a node model and chip to chip, random widths and polynomials looped chip to chip; formal proof of the CRC unit
22-bit `DIV`, `BND`, `SEEK`, `BYTE` and `SPAN` CRCs, release while reading v0.7	In RTL, tested	RC5 at 1.778 ms bits, K-Line 5-baud init, DALI received by the Manchester decoder, LIN header with a gap, Modbus and SMBus CRCs checked on chip, 3-wire SPI and MDIO push-pull; sixteen standards in `t_standards`
NRZI single-ended, Manchester Thomas	In RTL, not tested	Same paths as the tested codes

Open questions:

1. v0.5 I^2C target mode and capture-any headers, open in v0.4, are now implemented with `BRANCH` and `ANY`.
2. The return link carries two bytes per host entry. Full-size 10BASE-T frames (64 bytes and up) arrive faster than even a 4-clock link can return them; they are reported as overflows.
3. v0.5 Transmit packets are no longer limited by the 8-entry host FIFO, since `DATA` streams during the frame; the instruction link rate now sets how fast a fixed-timing line can be fed.
4. A host entry waiting for the return link holds up the wire entries behind it; I^2C just stretches SCL, but a timed transmit would pause, and an I^2C target above 1 MHz can answer late after a repeated START. v0.6 The CRC and the reply are exempt: they reach the line codec directly.